

UE24CS352A Machine Learning Mini-Project

Teammate 1 Complete Study & Code Explanation Guide

Supervised Pulsar Candidate Classification (GDA, Random Forest, 5-Fold CV & Threshold Tuning)

1. High-Level Project Overview & Problem Statement

Astronomical Problem: Radio pulsar search surveys (such as the High Time Resolution Universe Survey, HTRUS) collect millions of signal candidates. However, the vast majority are spurious noise or Radio Frequency Interference (RFI) from human technology (e.g. FM radio, radar). Scientists need machine learning to filter candidates automatically.

HTRU2 Dataset Breakdown:

- Total Candidate Signals: **17,898**
- Spurious Noise / RFI (Class 0): **16,259 (90.84%)**
- True Pulsars (Class 1): **1,639 (9.16%)**
- Imbalance Ratio: ~1:9.9 (For every 1 true pulsar, there are nearly 10 non-pulsars).
- **Features (8 continuous variables):**
 - 1-4: Mean, Standard Deviation, Excess Kurtosis, and Skewness of the *Integrated Pulse Profile*.
 - 5-8: Mean, Standard Deviation, Excess Kurtosis, and Skewness of the *DM-SNR Curve*.

2. Mathematical Formulations & Algorithms Explained

2.1 Gaussian Discriminant Analysis (GDA Baseline from Scratch)

GDA is a **generative classifier** that models the probability distribution of features $\mathbf{x}$ given class y , i.e. $P(\mathbf{x}|y)$. It assumes that $P(\mathbf{x}|y=0)$ and $P(\mathbf{x}|y=1)$ follow multivariate Gaussian distributions with individual mean vectors μ_0 and μ_1 , but share a common covariance matrix Σ .

1. Class Prior Probability (Bernoulli distribution):

$$\phi = P(y=1) = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(y^{(i)} = 1)$$

2. Class Mean Vectors:

$$\mu_0 = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(y^{(i)}=0) \mathbf{x}^{(i)} \quad \mu_1 = \frac{1}{N} \sum_{i=1}^N \mathbb{1}(y^{(i)}=1) \mathbf{x}^{(i)}$$

3. Shared Covariance Matrix Σ :

$$\Sigma = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}^{(i)} - \mu_{y^{(i)}})(\mathbf{x}^{(i)} - \mu_{y^{(i)}})^T$$

4. Classification Probability $P(y=1|\mathbf{x})$ (Bayes Rule):

By applying Bayes' Rule, the posterior log-likelihood ratio simplifies to a linear decision boundary:

$$P(y=1|\mathbf{x}) = \frac{1}{1 + \exp(-(Q_1(\mathbf{x}) - Q_0(\mathbf{x})))}$$

where $Q_k(\mathbf{x}) = -\frac{1}{2}(\mathbf{x} - \mu_k)^T \Sigma^{-1}(\mathbf{x} - \mu_k) + \log P(y=k)$.

2.2 Random Forest & Decision Trees (From Scratch)

Random Forest is an **ensemble algorithm** that combines multiple decision trees to reduce variance (overfitting) via **Bagging (Bootstrap Aggregation)** and **Random Feature Subsets**.

1. Shannon Entropy $E(S)$: Measures impurity in a dataset subset S :

$$E(S) = -p_0 \log_2(p_0) - p_1 \log_2(p_1)$$

2. Information Gain (IG): Evaluates the quality of a feature split at a node:

$$IG = E(\text{parent}) - \left(\frac{N_{\text{left}}}{N} E(\text{left}) + \frac{N_{\text{right}}}{N} E(\text{right}) \right)$$

3. Random Feature Subsets: At every node split, only a random subset of $m = \lfloor \sqrt{p} \rfloor = \lfloor \sqrt{8} \rfloor = 2$ features is considered.

4. Bootstrap Aggregation: Each tree is trained on a random sample of size N drawn with replacement from the training set.

3. Code Module Architecture & Line-by-Line Breakdown

■ `src/eda.py`

Handles dataset loading, class imbalance calculation, KDE probability density plots, and feature correlation matrix heatmap.

■ `src/gda.py`

Custom GaussianDiscriminantAnalysis class. `fit(X, y)` computes ϕ , μ_0 , μ_1 , and σ using NumPy matrix operations (`X.T @ X / n`). `predict_proba(X)` calculates Mahalanobis distances and sigmoid probability.

■ `src/random_forest_scratch.py`

DecisionTreeScratch & RandomForestScratch classes. Calculates Shannon entropy and Information Gain for continuous feature thresholds, uses recursive node splitting, and averages tree predictions.

■ `src/random_forest_sklearn.py`

Wraps `RandomForestClassifier` with `RandomizedSearchCV` over `n_estimators`, `max_depth`, `max_features`, `min_samples_split`, and `criterion`, optimizing for Recall.

■ `src/evaluation.py`

Includes `upsample_positive_class()` for random oversampling, `run_5fold_cross_validation()` with strict fold-level upsampling, `plot_threshold_sweep()` to shift boundary from 0.5 to 0.2, and ROC/PRC plotting functions.

■ `main_supervised.py`

Main execution entry point coordinating EDA, 80/20 train/test split, GDA baseline, Scikit-Learn RF tuning, Scratch RF, threshold sweeps, and JSON metrics export.

4. 5-Fold Cross-Validation & Data Leakage Prevention

Why Upsample? Due to the 1:9.9 class imbalance, un-sampled models tend to predict the majority class (non-pulsars) most of the time, leading to poor recall on pulsar candidates.

The Data Leakage Trap (CRITICAL): If you upsample the entire dataset *before* splitting into train/test or cross-validation folds, identical synthetic copies of the positive examples will appear in both the training set and the validation/test set. This causes artificially inflated 'fake' 99%+ accuracy.

Our Correct Protocol:

1. Split data into 80% Train and 20% Held-out Test.
2. Divide the training set into 5 folds using `StratifiedKFold`.
3. In each fold, upsample the positive class in the **training fold ONLY** (to ~50% positive).
4. Evaluate the model on the **validation fold WITHOUT upsampling** (raw 1:9.9 distribution).
5. Finally, evaluate on the untouched test set.

5. Threshold Sweep Optimization (0.5 → 0.2 Boundary Shift)

In standard binary classification, a decision threshold of 0.5 is used. However, in astronomical signal search:

- **False Negative (FN):** Discarding a true pulsar star candidate. (*EXTREMELY HIGH SCIENTIFIC COST* - lost discovery).
- **False Positive (FP):** Flagging noise as a pulsar candidate. (*LOW COST* - astronomers will briefly inspect it).

Following Section 7.2 of the CS229 paper (Debesai et al.), we performed a threshold sweep from 0.0 to 1.0. Lowering the decision threshold to **0.2** raised recall from **88.41%** → **91.16%** (Scikit-Learn RF) and **90.85%** → **93.29%** (Scratch RF).

6. viva & Presentation Q&A; Preparation (Questions & Answers)

Q1: Why did you implement GDA as a baseline instead of Naive Bayes or Logistic Regression?

GDA is a generative model designed for continuous features. Naive Bayes assumes features are independent given class, whereas GDA models feature dependencies using a full covariance matrix Σ . Logistic Regression is discriminative; GDA makes stronger multivariate normal assumptions which are asymptotically efficient when feature distributions are near-normal.

Q2: How did you ensure your cross-validation results did not suffer from data leakage?

We used fold-level upsampling. Inside each of the 5 cross-validation folds, Random Minority Oversampling was applied to the training fold ONLY. The validation fold was kept raw and untouched. This guaranteed zero synthetic data leakage into validation or testing.

Q3: What formulas did you use for your GDA implementation from scratch?

We calculated prior $\phi = N1/N$, class means $\mu_0 = \sum x_i(y=0)/N0$ and $\mu_1 = \sum x_i(y=1)/N1$, and shared covariance matrix $\Sigma = (1/N) \sum (x_i - \mu_{yi})(x_i - \mu_{yi})^T$. Posterior $P(y=1|x)$ was computed using the sigmoid of the log likelihood ratio: $1 / (1 + \exp(-(Q1 - Q0)))$.

Q4: How does your Random Forest from scratch split nodes?

At each node, it calculates Shannon Entropy $E(S) = -p_0 \log_2(p_0) - p_1 \log_2(p_1)$ and Information Gain $IG = E(\text{parent}) - (N_{\text{left}}/N E_{\text{left}} + N_{\text{right}}/N E_{\text{right}})$. It randomly selects $\sqrt{p}$ features and evaluates percentiles to find the feature and threshold that maximize Information Gain.

Q5: Why did you lower the decision threshold from 0.5 to 0.2?

In pulsar search, discarding a true pulsar (False Negative) is far worse than inspecting a noise candidate (False Positive). Lowering the decision threshold to 0.2 maximizes Recall (Sensitivity), raising pulsar candidate detection to 91.16%-93.29%.

Q6: What hyperparameters did RandomizedSearchCV select for Random Forest?

It selected `n_estimators=200`, `min_samples_split=2`, `max_features='log2'`, `max_depth=110`, and `criterion='entropy'`, optimizing directly for Recall score.

Q7: How does Teammate 1's output connect to Teammate 2's work?

Teammate 1 filters raw candidate signals and isolates true pulsar instances (1,639 positive candidates). Teammate 2 takes these positive pulsar candidates to run unsupervised clustering (K-means, Self-Organizing Map, PCA visualization) and demo app.